

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

05 - Mergesort

Prof. Alex Kutzke

Maio de 2021

Slides adaptados do material produzido
pelo Prof. Reinaldo Fortes

Roteiro

Introdução

Mergesort

Implementação Recursiva

Implementação Iterativa

Otimizações práticas

Conclusão

Prática

Motivação

- É preciso resolver um problema com uma entrada grande;
- Para facilitar a resolução do problema, a entrada é quebrada em pedaços menores (divisão);
- Cada pedaço da entrada é então tratado separadamente (conquista);
- Ao final, os resultados parciais são combinados para gerar o resultado final procurado.

Motivação

A técnica de **divisão e conquista** consiste de **3 passos**:

Divisão : dividir o problema original em subproblemas menores;

Conquista : resolver cada subproblema (talvez recursivamente);

Combinação : combinar as soluções encontradas, compondo uma solução para o problema original.

A técnica

- Algoritmos baseados em divisão e conquista são, em geral, **recursivos** (mas não necessariamente);
- A maioria dos algoritmos de divisão e conquista divide o problema em subproblemas da mesma natureza, de tamanho $\frac{n}{b}$;
- Vantagens:
 - **Uso eficiente da memória cache:** ao final da fase de divisão grande parte dos dados necessários para a fase de combinação já estão disponíveis na cache:
 - Com isso, requerem um número menor de acessos à memória (lembrem de S.O.)?
 - **São altamente paralelizáveis:** se existirem vários processadores disponíveis, a estratégia propiciará eficiência;

Quando utilizar?

Existem três condições que indicam que a estratégia de divisão e conquista pode ser utilizada com sucesso:

- Deve ser possível decompor uma instância em sub-instâncias;
- A combinação dos resultados deve ser eficiente (trivial, se possível);
- As sub-instâncias devem ser mais ou menos do mesmo tamanho.

Algoritmo genérico

Function divisao_e_conquista(x):

if x *pequeno ou simples* **then**

return x

else

 decompor x em n conjuntos menores: $x[0], x[1], \dots, x[n-1]$

for i *in* $[0, 1, \dots, n-1]$ **do**

$y[i] = \text{divisao_e_conquista}(x[i])$

end

 combinar $y[0], y[1], \dots, y[n-1]$ em y

return y

end

Utilização na ordenação

- Métodos de ordenação que utilizam-se da **Divisão e Conquista**:
 - Mergesort, Quicksort e vários outros;
- Principal diferença entre Mergesort e Quicksort:
 - **Mergesort**: sempre divide o problema de forma balanceada (subproblemas de mesmo tamanho);
 - **Quicksort**: utiliza o conceito de **pivô** para dividir o problema em subproblemas (subproblemas de tamanhos diferentes);

Algoritmo

1. Divida a coleção em duas partes;
2. Ordene as duas partes utilizando o mesmo Algoritmo;
3. Intercale (merge) as duas partes ordenadas, obtendo uma coleção ordenada.

Funcionamento

A execução do **Mergesort** pode ser facilmente descrita por uma espécie de árvore (binária):

- Cada nó representa uma chamada recursiva do **Mergesort**;
- O nó raiz é a chamada inicial;
- Os nós folha são vetores de 1 ou 2 números (caso base).

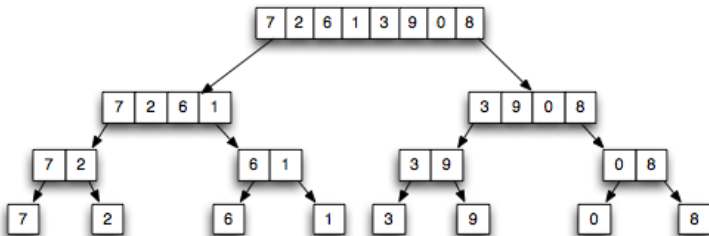


Figura: Exemplo divisões mergesort. Fonte:

<<http://www.codenlearn.com/2011/10/simple-merge-sort.html>>

Execução

Utilize o Mergesort para ordenar a seguinte coleção de números:

0	1	2	3	4	5	6
38	27	43	3	9	82	10

Execução

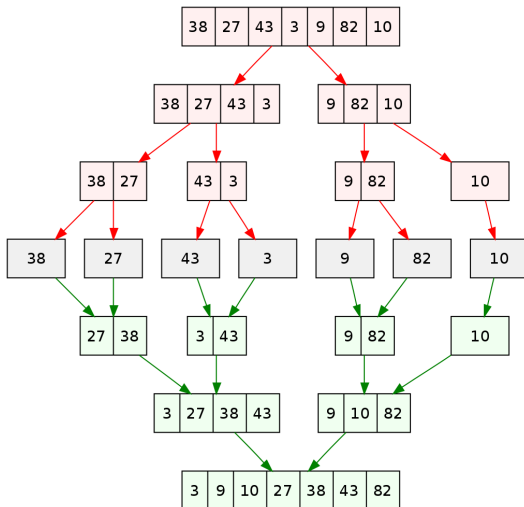


Figura: Exemplo execução mergesort. Fonte:

<https://en.wikipedia.org/wiki/Merge_sort>

Mergesort Recursivo

```
1 void mergeSort(int *v, int n) {
2     mergeSort_ordena (v, 0, n-1);
3 }
4
5 void mergeSort_ordena (int *v, int esq , int dir) {
6     if (esq == dir)
7         return;
8
9     int meio = (esq + dir) / 2;
10    mergeSort_ordena (v, esq , meio);
11    mergeSort_ordena (v, meio+1, dir);
12    merge (v, esq , meio , dir);
13 }
```

Merge

```
1 void merge(int * a, int l, int m, int r)
2 {
3     int i,j,k, *aux;
4     aux = (int *) malloc(sizeof(int) * r+1);
5
6     for(i=m+1; i > l; i--) aux[i-1] = a[i-1];
7     for(j=m; j < r; j++) aux[r+m-j] = a[j+1];
8
9     for(k=l; k<=r; k++)
10         if(aux[j] < aux[i]) a[k] = aux[j--];
11         else a[k] = aux[i++];
12 }
```

Análise empírica

Estimativas:

- Laptop executa 10^8 comparações por segundo;
- Supercomputador executa 10^{12} comparações por segundo;

	insertion sort (N^2)			mergesort ($N \log N$)		
computer	thousand	million	billion	thousand	million	billion
home	instant	2.8 hours	317 years	instant	1 second	18 min
super	instant	1 second	1 week	instant	instant	instant

Figura: Tabela de comparação de Insertsort e Mergesort. Fonte: Adaptado de (SEGEWICK; WAYNE, 2014)

Conclusão?

Análise empírica

Estimativas:

- Laptop executa 10^8 comparações por segundo;
- Supercomputador executa 10^{12} comparações por segundo;

computer	insertion sort (N^2)			mergesort ($N \log N$)		
	thousand	million	billion	thousand	million	billion
home	instant	2.8 hours	317 years	instant	1 second	18 min
super	instant	1 second	1 week	instant	instant	instant

Figura: Tabela de comparação de Insertsort e Mergesort. Fonte: Adaptado de (SEdgeWICK; WAYNE, 2014)

Conclusão?

Bons algoritmos são melhores do que supercomputadores.

Análise Mergesort Recursivo

Vantagens

- Mergesort é $O(n \log_2 n)$;
- Indicado para aplicações que tem restrição de tempo (executa sempre em um determinado tempo para n);
- Passível de ser transformado em estável:
 - Tomado certos cuidados na implementação da intercalação.
- Fácil implementação.

Desvantagens

- Utiliza memória adicional, em $O(n)$:
 - Existem implementações que não necessitam de espaço adicional, mas são complexas;
- Na prática é mais lento que **Quicksort** no caso médio.

Mergesort Iterativa

Você consegue implementar a versão iterativa do Mergesort?
Qual a complexidade da sua implementação?

Possíveis otimizações

Embora o **Mergesort** seja considerado como assintoticamente ótimo¹, algumas otimizações simples podem ser realizadas para deixá-lo ainda mais eficiente. Dentre elas:

- Utilizar Insertsort em partes pequenas do array;
- Não realizar a operação merge caso a coleção já esteja ordenada;
- Eliminar a cópia para um array auxiliar (poupa tempo mas não espaço);
- Aproveitar trechos pré-ordenados;

Mas lembre-se, embora essas otimizações tornem o algoritmo mais eficiente em alguns casos, sua complexidade ainda é $O(n \log_2 n)$ no pior caso.

¹O número de comparações realizado pelo Mergesort no pior caso é o menor número de comparações possível para qualquer algoritmo de ordenação *baseado em comparações* (SEDGEWICK; WAYNE, 2014).

Insertsort para pequenas porções

- Mergesort tem muito overhead para arrays pequenos;
- Determinar uma linha de corte (*cutoff*): trechos com um número menor de elementos do que a linha de corte são ordenados por Insertsort;

Insertsort para pequenas porções

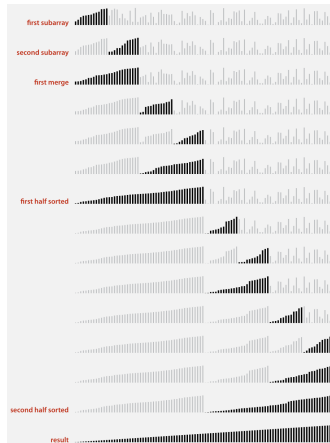


Figura: Mergesort com cutoff para insertion sort. Fonte: (SEdgeWICK; WAYNE, 2014)

Impedir merge caso seja desnecessário

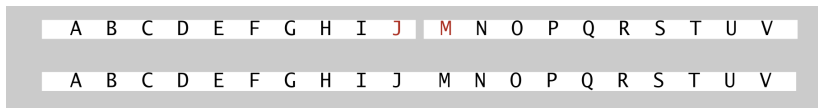


Figura: Mergesort sem merge desnecessário. Fonte: (SEdgeWICK; WAYNE, 2014)

```
1 // ...
2 mergesort(a, l, m);
3 mergesort(a, m+1, r);
4 if(a[m] <= a[m+1]) return;
5 // ...
```

Mergesort natural

- Aproveitar trechos já ordenados da coleção:

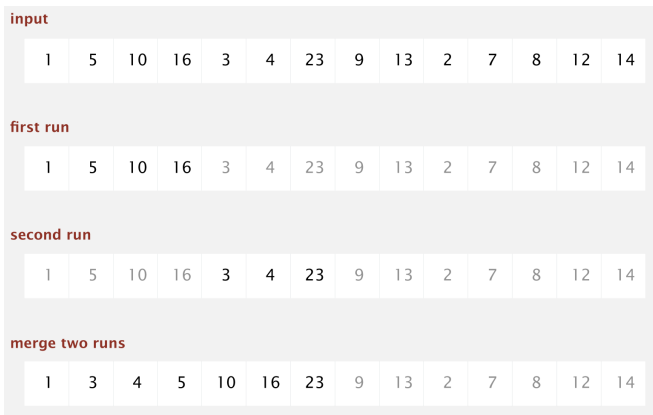


Figura: Mergesort natural. Fonte: (SEdgeWICK; WAYNE, 2014)

Algoritmos de ordenação

	inplace?	stable?	best	average	worst	remarks
selection	✓		$\frac{1}{2} N^2$	$\frac{1}{2} N^2$	$\frac{1}{2} N^2$	N exchanges
insertion	✓	✓	N	$\frac{1}{4} N^2$	$\frac{1}{2} N^2$	use for small N or partially ordered
merge		✓	$\frac{1}{2} N \lg N$	$N \lg N$	$N \lg N$	$N \log N$ guarantee; stable
timsort		✓	N	$N \lg N$	$N \lg N$	improves mergesort when preexisting order
?	✓	✓	N	$N \lg N$	$N \lg N$	holy sorting grail

Figura: Tabela de comparação de algoritmos de ordenação. Fonte: Adaptado de (SEGEWICK; WAYNE, 2014)

Curiosidade: algoritmo chamado **Timsort** (que é a implementação de algumas das otimizações apresentadas) é o algoritmo utilizado em algumas bibliotecas padrão de linguagens e ambientes como: Python, Java 7, GNU Octave, Android, ...

Conclusão

- Algoritmo assintoticamente ótimo para o problema da ordenação geral;
- Fácil implementação;
- Otimizações práticas;

Exercícios sugeridos

Além de implementar a versão iterativa do Mergesort, realize o seguinte exercício para fixação do algoritmo:

- Dada a sequência de números: $[3, 4, 9, 2, 5, 1, 8]$, ordene em ordem crescente utilizando o algoritmo Mergesort passo-a-passo (teste de mesa).

